

Introducción a los lenguajes de scripting de cliente

¿Qué son los lenguajes de scripting de cliente?

Los **lenguajes de scripting de cliente** son lenguajes de programación que se ejecutan en el **navegador web del usuario** (el "cliente"), a diferencia de los lenguajes de servidor que se ejecutan en el servidor web antes de enviar el contenido al usuario.

Estos lenguajes permiten crear páginas web **dinámicas** e **interactivas**, capaces de responder a las acciones del usuario, modificar el contenido de la página, validar formularios, crear animaciones y mucho más, todo ello sin necesidad de recargar la página completa.

Características principales

1. Ejecución en el cliente

- El código se descarga junto con la página web
- Se ejecuta directamente en el navegador del usuario
- No requiere comunicación con el servidor para ejecutarse
- La velocidad de ejecución depende del dispositivo del usuario

2. Interactividad

- Permite responder a eventos del usuario (clics, teclas, movimientos del ratón)
- Modifica el contenido de la página en tiempo real
- Crea experiencias de usuario más fluidas y dinámicas

3. Acceso al DOM (Document Object Model)

- Pueden acceder y modificar la estructura HTML de la página
- Permiten cambiar estilos CSS dinámicamente
- Facilitan la creación y eliminación de elementos HTML

En apartados posteriores explicaremos con detalle qué es el DOM y cómo manipularlo.

4. Interpretado

- No necesitan compilación previa

- Se interpretan línea a línea durante la ejecución
- Los errores se detectan en tiempo de ejecución

Diferencias entre scripting de cliente y servidor

Aspecto	Cliente	Servidor
Lugar de ejecución	Navegador del usuario	Servidor web
Velocidad	Depende del dispositivo del usuario	Depende del servidor
Seguridad	Código visible para el usuario	Código oculto al usuario
Acceso a datos	Limitado por el navegador	Acceso completo a bases de datos
Recarga de página	No necesaria	Generalmente necesaria
Ejemplos	JavaScript, TypeScript	PHP, Python, Java, C#

Ventajas de los lenguajes de cliente

Interactividad inmediata

- Respuesta instantánea a las acciones del usuario
- No hay tiempo de espera para comunicarse con el servidor
- Experiencia de usuario más fluida

Reducción de carga del servidor

- Muchas tareas se ejecutan en el cliente
- Menor número de peticiones al servidor
- Mejor rendimiento general del sitio web

Funcionalidad offline

- Algunas funcionalidades pueden trabajar sin conexión
- Validación de formularios sin conexión a internet
- Almacenamiento local de datos

Interfaz dinámica

- Modificación del contenido sin recargar la página
- Animaciones y efectos visuales
- Actualización de partes específicas de la página

Desventajas de los lenguajes de cliente

Dependencia del navegador

- Diferencias de compatibilidad entre navegadores
- El usuario puede desactivar JavaScript
- Limitaciones de seguridad del navegador

Seguridad limitada

- El código es visible para cualquier usuario
- No se pueden realizar operaciones sensibles
- Validaciones deben repetirse en el servidor

Limitaciones de acceso

- No pueden acceder directamente a bases de datos
- No pueden escribir archivos en el servidor
- Restricciones de acceso a recursos externos (CORS)

Dependencia del dispositivo

- Rendimiento variable según el dispositivo del usuario
- Consumo de batería en dispositivos móviles
- Limitaciones de memoria y procesamiento

Principales lenguajes de scripting de cliente

A lo largo de los años han surgido diferentes lenguajes de scripting de cliente, aunque no todos han tenido el mismo éxito o adopción. A continuación, se presentan los más relevantes:

JavaScript

- **El más utilizado** y prácticamente el estándar

- Soportado por todos los navegadores modernos
- Gran ecosistema de librerías y frameworks
- Evolución constante con nuevas versiones (ES6, ES2017, etc.)

TypeScript

- Superconjunto de JavaScript con **tipado estático**
- Compilado a JavaScript
- Mejor para proyectos grandes y equipos
- Desarrollado por Microsoft

Otros lenguajes

- **Dart**: Desarrollado por Google, compila a JavaScript
- **CoffeeScript**: Sintaxis simplificada que compila a JavaScript
- **WebAssembly**: Para código de alto rendimiento

LIBRERÍAS Y FRAMEWORKS

En el contexto del desarrollo de software es muy común el uso de **librerías** y **frameworks** que facilitan y aceleran el desarrollo con estos lenguajes.

Llamamos **librería** a un conjunto de funciones y utilidades que podemos usar en nuestro código para realizar tareas comunes sin tener que escribirlas desde cero. Por ejemplo, **jQuery** es una librería muy popular que simplifica la manipulación del DOM y la gestión de eventos en JavaScript.

Por su parte, un **framework** es una estructura más completa que proporciona **una base** sobre la cual construir aplicaciones. Un framework define la **arquitectura** de la aplicación y suele incluir **herramientas para gestionar el flujo** de datos, la interfaz de usuario y la comunicación con el servidor. Ejemplos populares de frameworks para JavaScript son **React**, **Angular** y **Vue.js**.

LENGUAJES DE PROGRAMACIÓN TIPADOS Y NO TIPADOS

En programación, los lenguajes pueden clasificarse según si son **tipados** o **no tipados** (también llamados **dinámicamente tipados**).

- **Lenguajes tipados**: En estos lenguajes, cada variable tiene un **tipo de dato específico** (como entero, cadena, booleano, etc.) que debe ser declarado

explícitamente. Esto permite **detectar errores de tipo** en tiempo de compilación. Ejemplos de lenguajes tipados son **TypeScript, Java y C#**.

- Entre los lenguajes tipados, podemos distinguir entre:
 - **Tipado estático:** El tipo de dato se verifica en tiempo de compilación (ej. TypeScript, Java).
 - **Tipado fuerte:** El lenguaje impone reglas estrictas sobre cómo se pueden combinar diferentes tipos de datos (ej. Python, Ruby).
 - **Tipado débil:** El lenguaje permite conversiones implícitas entre tipos de datos (ej. JavaScript, PHP).
- **Lenguajes no tipados:** En estos lenguajes, las variables no tienen un tipo fijo y pueden cambiar de tipo durante la ejecución. Esto ofrece **mayor flexibilidad**, pero puede llevar a errores que solo se detectan en tiempo de ejecución. Ejemplos de lenguajes no tipados son **JavaScript, Python y Ruby**.

JavaScript: El protagonista

JavaScript es, sin duda, el lenguaje de scripting de cliente más importante y utilizado. Aunque inicialmente fue creado para añadir pequeñas interacciones a las páginas web, hoy en día es un lenguaje completo y potente que permite:

- Desarrollo de aplicaciones web complejas
- Aplicaciones móviles (React Native, Ionic)
- Aplicaciones de escritorio (Electron)
- Desarrollo del lado servidor (Node.js)
- Desarrollo de videojuegos
- Inteligencia artificial y machine learning

Historia y evolución

1995: Los inicios

- Brendan Eich crea JavaScript en Netscape en solo 10 días
- Originalmente llamado "LiveScript"
- Diseñado para hacer las páginas web más dinámicas

1997: Estandarización

- Nace ECMAScript (ES1) como estándar

- Compatibilidad entre diferentes navegadores

2009: ES5

- Muchas mejoras y nuevas características
- Mejor soporte en navegadores

2015: ES6/ES2015

- Revolución en JavaScript
- Clases, módulos, arrow functions, let/const
- Ciclo de actualizaciones anuales

2016-presente: ES2016, ES2017...

- Nuevas características cada año
- async/await, destructuring, spread operator
- JavaScript moderno y potente

Aplicaciones modernas

Los lenguajes de scripting de cliente han evolucionado mucho más allá de simples validaciones de formularios:

Single Page Applications (SPA)

- Aplicaciones que cargan una sola vez
- Navegación fluida entre secciones
- Frameworks como React, Angular, Vue.js

Progressive Web Apps (PWA)

- Aplicaciones web que funcionan como apps nativas
- Funcionamiento offline
- Instalables en dispositivos móviles

Aplicaciones en tiempo real

- Chat en vivo

- Colaboración en documentos
- Juegos multijugador

¿Por qué aprender scripting de cliente?

1. **Demanda laboral:** JavaScript es uno de los lenguajes más demandados
2. **Versatilidad:** Un lenguaje para múltiples plataformas
3. **Comunidad:** Gran comunidad y recursos de aprendizaje
4. **Evolución:** Constante mejora y nuevas posibilidades
5. **Creatividad:** Permite crear experiencias de usuario innovadoras

En este tema aprenderemos...

Para aprender y dominar Javascript, necesitaríamos muchas más horas de las que disponemos en este módulo para ello. Por lo tanto, en esta unidad didáctica nos centraremos en los fundamentos y los aspectos más relevantes y prácticos para que puedas empezar a utilizar JavaScript en tus proyectos web. Concretamente, aprenderemos a:

- **Programar en JavaScript** desde cero (especialmente orientado a ASIR)
- **Manipular el DOM** para modificar páginas web
- **Gestionar eventos** para crear interactividad
- **Modificar estilos** dinámicamente
- **Crear proyectos prácticos** que demuestren estas habilidades